

Artificial Intelligence

Chapter 3 (part 2): Heuristic Search

Adapted from slides available in Russell & Norvig's textbook webpage

Outline

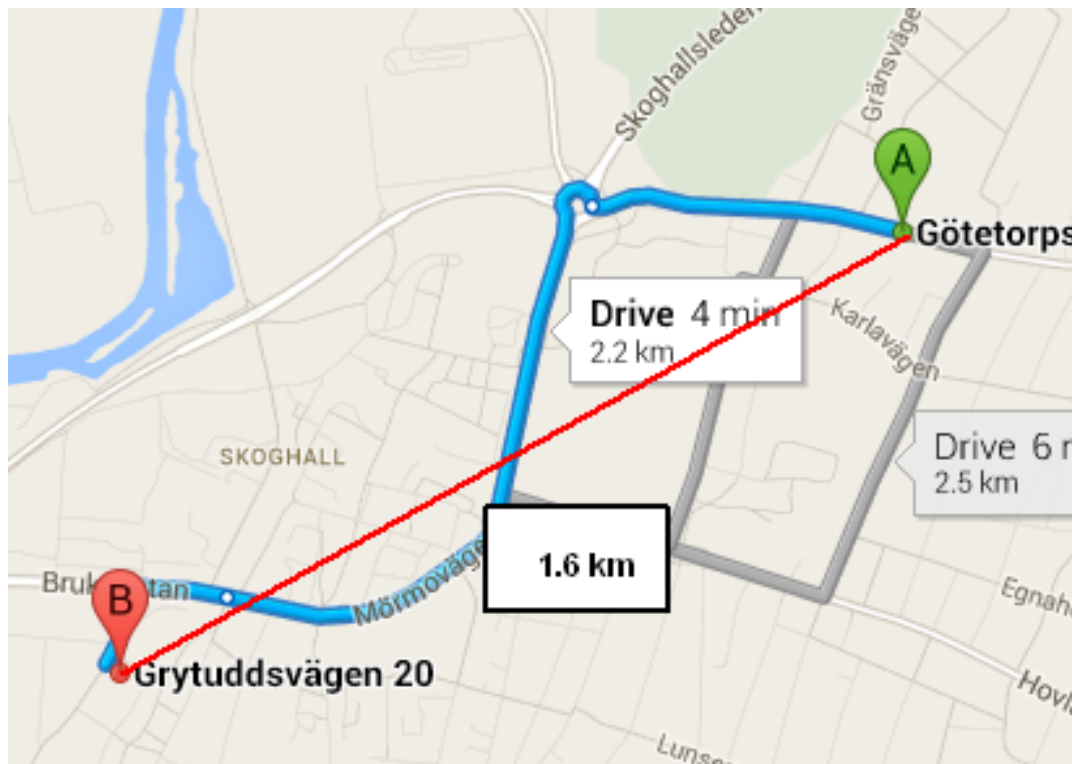
- Heuristics
- Best-first search
- Greedy best-first search
- A^* search
- More on heuristics

Definition of heuristics

- A **heuristic technique** (/hju:'ristɪk/; Ancient Greek: εὕρισκω, "find" or "discover"), often called simply a heuristic, is any approach to problem solving, learning, or discovery that employs a practical method not guaranteed to be optimal or perfect, but sufficient for the immediate goals.
- Heuristics can be mental shortcuts that ease the cognitive load of making a decision.
- Examples of this method include using a **rule of thumb**, an **educated guess**, or **common sense**.

Example: Driving from A to B

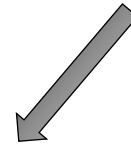
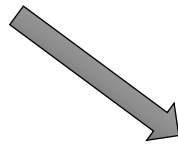
- The straight line distance is a heuristic to estimate the driving distance



Example: 8-puzzle problem

1	2	3
8		4
7	6	5

2	8	3
1	6	4
7		5



	1	2
3	4	5
6	7	8

Which state is “closer” to the goal state?

How can we quantify this?

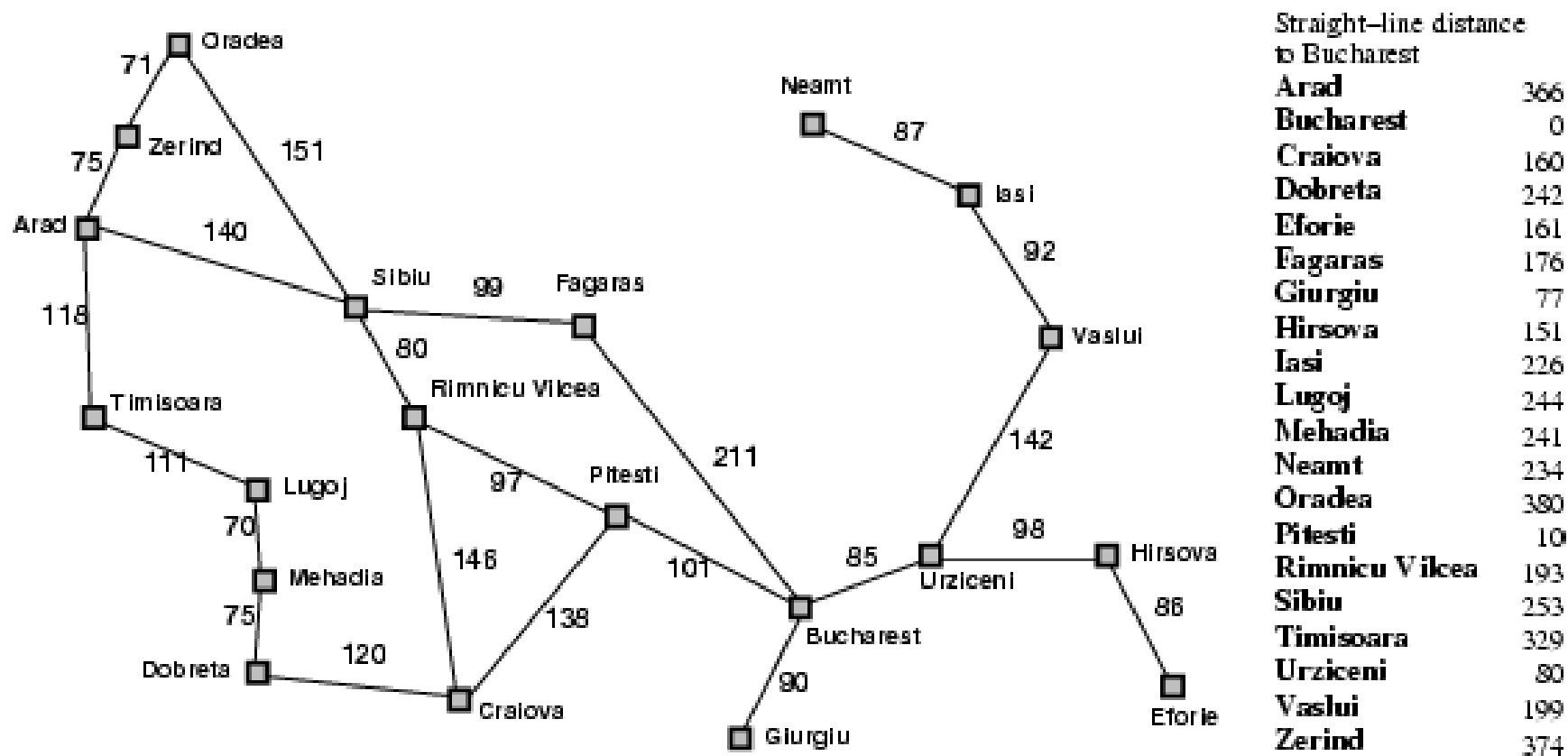
Best-first search

- Idea: use an **evaluation function** $f(n)$ for each node
 - estimate of "desirability"

→ Expand **most desirable** unexpanded node
- Implementation:

Order the nodes in fringe in decreasing order of desirability
- Special cases:
 - Greedy best-first search
 - A^* search

Romania with step costs in km



Greedy best-first search

- Evaluation function $f(n) = b(n)$ (**h**euristic)
= estimate of cost from n to *goal*
- e.g., $b_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy best-first search expands the node that **appears** to be closest to goal

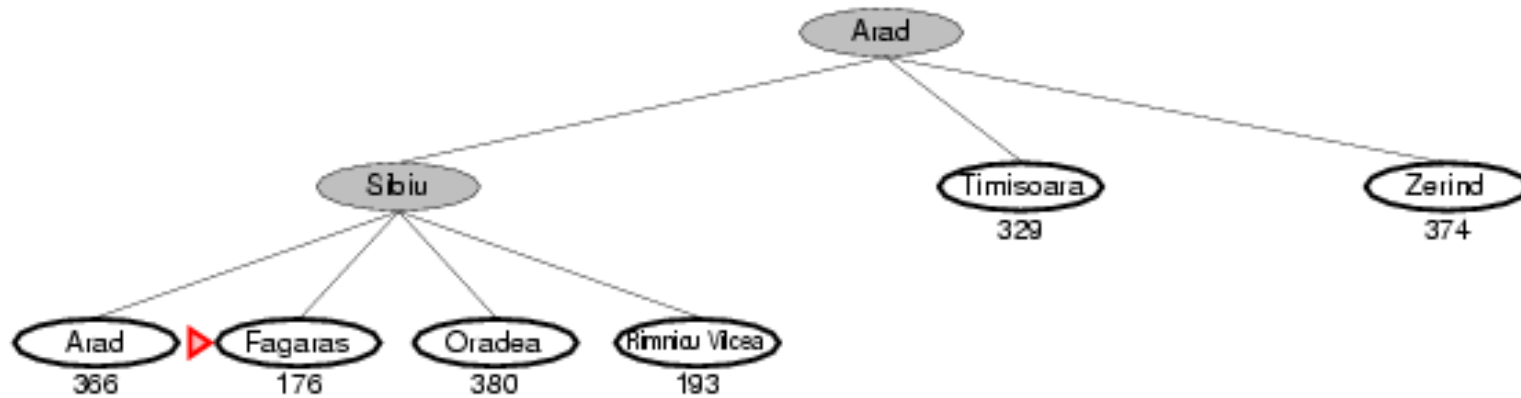
Greedy best-first search example



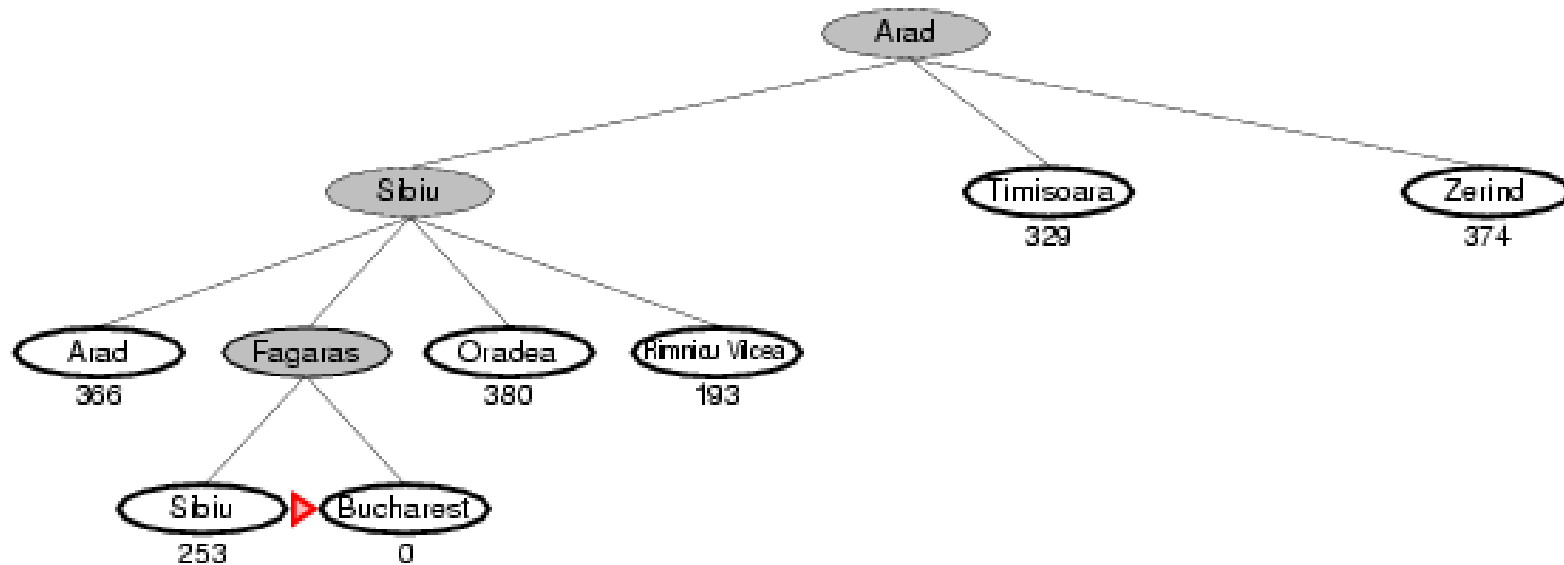
Greedy best-first search example



Greedy best-first search example



Greedy best-first search example



Properties of greedy best-first search

- Complete? No – can get stuck in loops, e.g., when going from Iasi to Fagars:

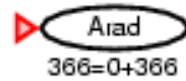
Iasi $\rightarrow$ Neamt $\rightarrow$ Iasi $\rightarrow$ Neamt $\rightarrow$

- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

A^* search

- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal

A^* search example



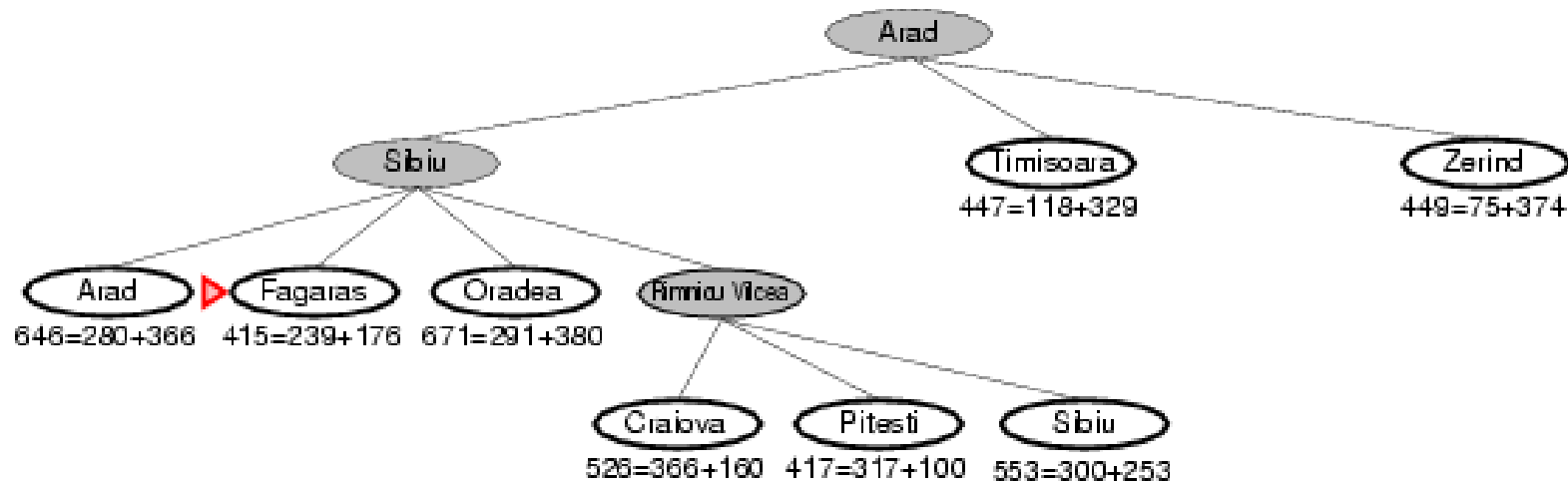
A* search example



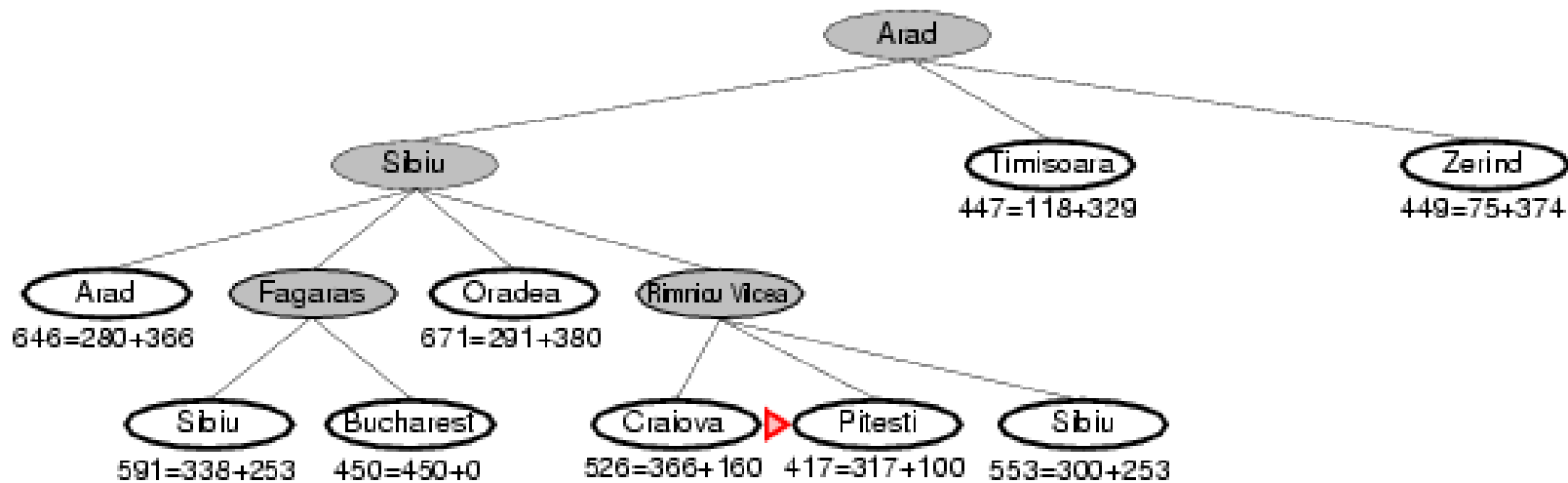
A* search example



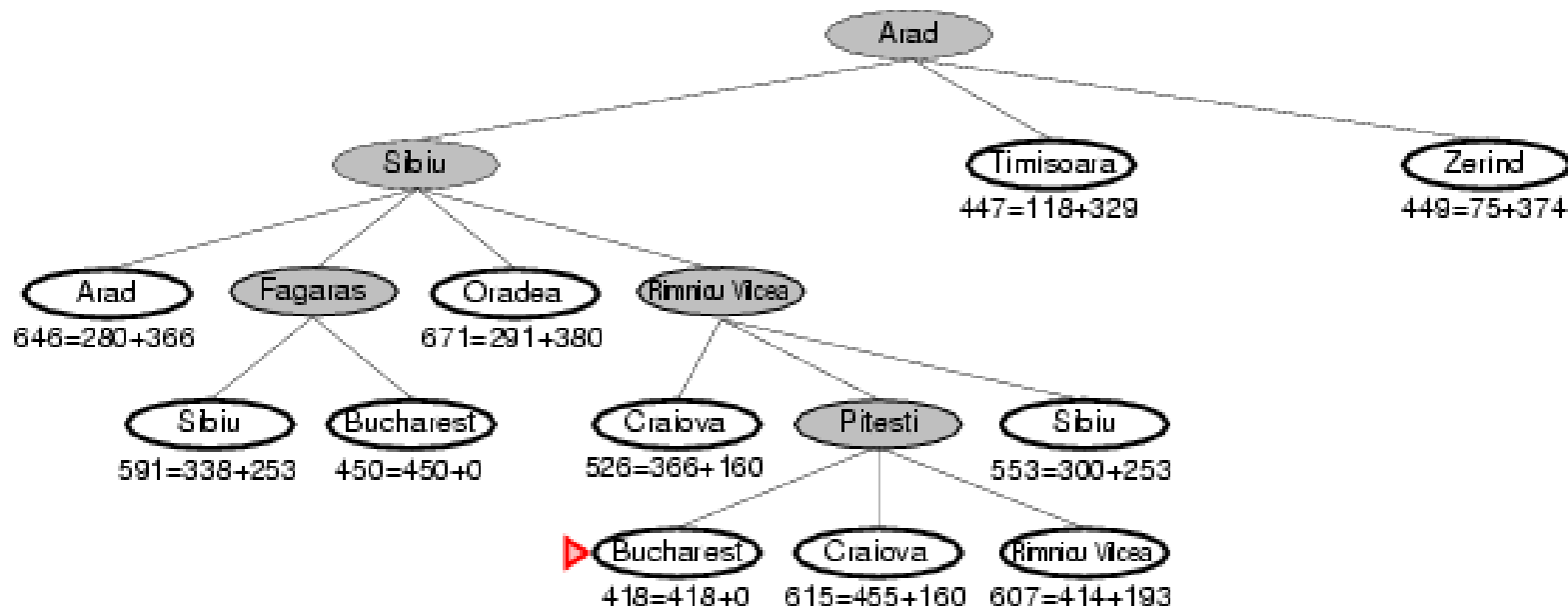
A* search example



A* search example



A* search example



Properties of A^*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes

You are given the following graph, where nodes represent cities, and edges represent the distance between them. The task is to find the shortest path from city **S** (Start) to city **G** (Goal) using the A* search algorithm. Also, write down the shortest path with the cost at the end. The graph is as follows and the heuristic (estimated distance to the goal, $h(n)$) for each node is given below.

From	To	Distance
S	A	2
S	B	4
A	C	4
A	D	7
B	D	1
B	E	3
C	G	2
D	G	1
E	G	5

From	Heuristic ($h(n)$)
S	7
A	6
B	4
C	2
D	1
E	5
G	0

1. Start at S:

- $f(S)=g(S)+h(S)=0+7=7$
 $f(S) = g(S) + h(S) = 0 + 7 = 7$
- Expand S, neighbors are:
 - **A:** $g(A)=g(S)+\text{distance}(S,A)=0+2=2$
 $g(A) = g(S) + \text{distance}(S, A) = 0 + 2 = 2$
 $2g(A)=g(S)+\text{distance}(S,A)=0+2=2$, $f(A)=g(A)+h(A)=2+6=8$
 $f(A) = g(A) + h(A) = 2 + 6 = 8$
 $8f(A)=g(A)+h(A)=2+6=8$.
 - **B:** $g(B)=g(S)+\text{distance}(S,B)=0+4=4$
 $g(B) = g(S) + \text{distance}(S, B) = 0 + 4 = 4$
 $4g(B)=g(S)+\text{distance}(S,B)=0+4=4$, $f(B)=g(B)+h(B)=4+4=8$
 $f(B) = g(B) + h(B) = 4 + 4 = 8$
 $8f(B)=g(B)+h(B)=4+4=8$.
- Open list: [A:f(A)=8,B:f(B)=8][A: f(A)=8, B: f(B)=8][A:f(A)=8,B:f(B)=8].
- Closed list: [S][S][S].

2. Select A (tie-breaking arbitrarily):

- Expand A, neighbors are:
 - **C:** $g(C)=g(A)+\text{distance}(A,C)=2+4=6$
 $g(C) = g(A) + \text{distance}(A, C) = 2 + 4 = 6$
 $6g(C)=g(A)+\text{distance}(A,C)=2+4=6$, $f(C)=g(C)+h(C)=6+2=8$
 $f(C) = g(C) + h(C) = 6 + 2 = 8$
 $8f(C)=g(C)+h(C)=6+2=8$.
 - **D:** $g(D)=g(A)+\text{distance}(A,D)=2+7=9$
 $g(D) = g(A) + \text{distance}(A, D) = 2 + 7 = 9$
 $9g(D)=g(A)+\text{distance}(A,D)=2+7=9$, $f(D)=g(D)+h(D)=9+1=10$
 $f(D) = g(D) + h(D) = 9 + 1 = 10$
 $10f(D)=g(D)+h(D)=9+1=10$.
- Open list: [B:f(B)=8,C:f(C)=8,D:f(D)=10][B: f(B)=8, C: f(C)=8, D: f(D)=10][B:f(B)=8,C:f(C)=8,D:f(D)=10].
- Closed list: [S,A][S, A][S,A].

3. Select B (tie-breaking again):

- Expand **B**, neighbors are:
 - D**: $g(D)=g(B)+\text{distance}(B,D)=4+1=5$
 $g(D) = g(B) + \text{distance}(B, D) = 4 + 1 = 5$
 $f(D)=g(D)+h(D)=5+1=6$
 $f(D) = g(D) + h(D) = 5 + 1 = 6$
(Update $f(D)$ as it's better.)
 - E**: $g(E)=g(B)+\text{distance}(B,E)=4+3=7$
 $g(E) = g(B) + \text{distance}(B, E) = 4 + 3 = 7$
 $f(E)=g(E)+h(E)=7+5=12$
 $f(E) = g(E) + h(E) = 7 + 5 = 12$
- Open list: [C:f(C)=8,D:f(D)=6,E:f(E)=12]
- Closed list: [S,A,B]

4. Select D:

- Expand **D**, neighbors are:
 - G**: $g(G)=g(D)+\text{distance}(D,G)=5+1=6$
 $g(G) = g(D) + \text{distance}(D, G) = 5 + 1 = 6$
 $f(G)=g(G)+h(G)=6+0=6$
 $f(G) = g(G) + h(G) = 6 + 0 = 6$
- Open list: [C:f(C)=8,E:f(E)=12,G:f(G)=6]
- Closed list: [S,A,B,D]

5. Select G:

Goal reached. The shortest path is found

Shortest Path

The path is $S \rightarrow B \rightarrow D \rightarrow G$ with a total cost of 6.